

Analyse, modélisation, documentation et conception : Combat de robots

Objectifs - Concevoir et développer une application console client-serveur en utilisant la méthode de communication par socket synchrone

- Votre application devra permettre à 2 joueurs de s'affronter dans un combat de robots en réseau (**à tour de rôle**) ;
 - Un des joueurs agit comme serveur et comme joueur; le second joueur se connecte à celui-ci en tant que client ;
 - Vous devrez pouvoir jouer sur 2 ordinateurs différents* (*application serveur sur un, le client sur un autre*).
- *Pour des raisons de sécurité réseau, ceci n'est pas nécessairement possible!**
- Vous devrez utiliser les mécanismes appropriés afin d'effectuer les communications entre le client et le serveur ;
 - Vous devrez permettre la saisie des informations afin d'établir correctement la connexion entre le client et le serveur (*adresse IP et port du serveur pour la connexion → **pas localhost***).

Date de remise (Projet final)

- À remettre au plus tard Vendredi le 25 Septembre à 23h59

Dates de remise intermédiaires (Livrables partiels)

- Plan de développement mercredi 16 septembre à 13h59
 - Analyse des besoins, calendrier/mini échéancier avec estimation, planification des tests;
 - Diagrammes : cas d'utilisation, de séquences, de classes.
- Tests unitaires principaux mercredi 16 septembre à 13h59

Remise

- L'ensemble des documents de conception et d'analyse (*ceux demandés*)
- Remettre sur LÉA un document Word contenant :
 - le lien GIT de votre dépôt Git
 - les noms des coéquipiers
 - La liste (titre) des documents produits
- Il est fortement suggéré d'utiliser GIT (*adéquatement*) tout au long de votre développement, pas seulement pour déposer le produit fini!

Directives générales

- Ce travail doit être réalisé en équipe (*Équipes définies par l'enseignant*)
 - *Utilisez les mêmes équipes que celle du travail précédent*
- La solution sera compilée sur un environnement standard VS 2026 Community
 - **Une solution qui ne compile pas = programme non fonctionnel = ZÉRO**

Critères d'évaluation (Sommaire)

- Fonctionnement du programme et qualité de la documentation (3)
- Utilisation adéquate des concepts de la POO, validations, gestion d'exceptions et tests (1)
- Modularité, structure et architecture du code (1)

Consignes / Contraintes

Vous devrez respecter les consignes et contraintes suivantes :

- Vous devez utiliser les sockets synchrones. (*Vous pouvez réutiliser votre projet effectué en Intro Services de Données en adaptant le tout pour les besoins du présent travail*) ;
- Votre programme doit gérer adéquatement (*autant que possible*) les exceptions et effectuer les validations nécessaires au bon fonctionnement des traitements du programme ;
 - Exemple :
 - Perte de connexion réseau;
 - Client qui ferme son application sans demander/valider avec le serveur;
 - Configuration de robot invalide ou action impossible;
 - Message réseau invalide, incomplet ou inattendu.
- Vous devez programmer vos implémentations en utilisant **les séquences produites en équipe** ;
- Le serveur est en attente continue d'une connexion client tant qu'une partie n'a pas été débutée (*à suite à la connexion d'un client*);
- Une fois la connexion établie entre un client et le serveur, les 2 joueurs devraient pouvoir effectuer autant de parties qu'ils le désirent sans rétablir une nouvelle connexion;
- Votre programme devra permettre la connexion client/serveur peu importe où les programmes client et serveur sont exécutés*. (*Paramétrer les informations « hôte » avant d'établir la connexion*) ;
 - *Attention aux chemins d'accès (*relatifs vs absolus*), emplacement de l'exécutable, etc... ;
- Le serveur agit comme autorité sur l'état officiel de la partie : il valide les configurations, les actions demandées et les résultats du combat ;
- Lorsque le client décide de cesser de jouer (*après une partie*), le serveur se remet en mode attente d'une nouvelle connexion ;
- Il ne peut y avoir qu'une seule connexion client concurrente sur le serveur.

Informations importantes en lien avec les consignes

- Ne pas confondre « Gestion d'erreur » et validations.
 - Éviter autant que faire se peut, l'utilisation de break et continue.
 - Toute utilisation devra être justifiée
- **Minimiser et standardiser les informations transmises** entre le client et le serveur.
 - Vous devez utiliser le principe de sérialisation vu la/les sessions précédentes

Livrable final du mini-projet #2

- Vous devez livrer tout le nécessaire afin d'avoir une application fonctionnelle client-serveur permettant de jouer au Combat de robots ;
- Après le projet, vous devrez présenter à l'enseignant votre projet et les différents documents de conception ;
- Votre projet devra contenir un projet de test afin de tester différents éléments de votre projet ;
 - Minimalelement ceux listés dans la documentation livrée le 16 septembre ;
- Garder en tête que vous devriez mettre en place **des tests** qui **valident les éléments cruciaux** de vos implémentations.

Exigences du programme (règles du jeu) - Combat de robots

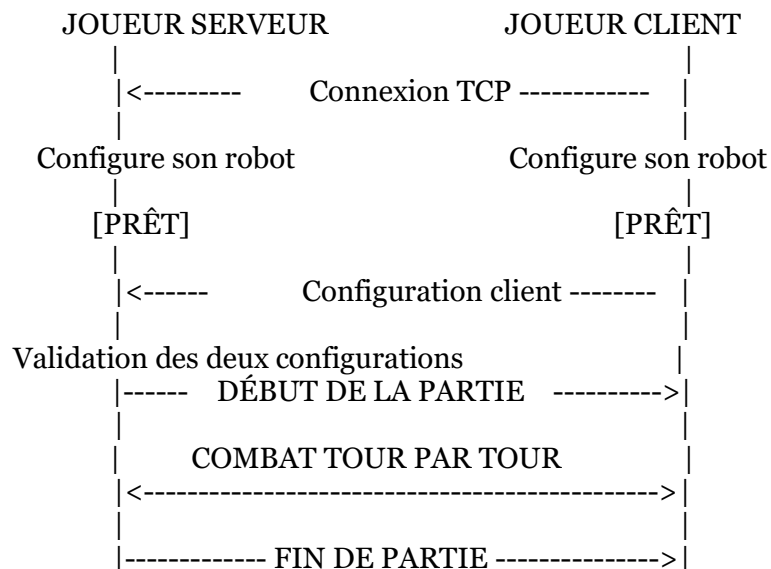
- Avant chaque partie, chaque joueur configure secrètement son robot en répartissant 10 points d'amélioration entre trois caractéristiques :
 - Points de vie (*HP*) ;
 - Armure ;
 - Dégâts.
- Valeurs de base des robots sont :
 - 100 HP ;
 - 0 point d'armure ;
 - 10 points de dégâts.
- Modification aux valeurs de base :
 - Chaque point investi en HP ajoute 10 HP ;
 - Chaque point investi en Armure ajoute 2 points de protection ;
 - Chaque point investi en Dégâts ajoute 2 points de dégâts.
- La configuration devient verrouillée dès qu'elle est confirmée. ;
 - Le serveur doit valider que l'entièreté des 10 points (*et seulement 10 points*) ont été répartis correctement.
- La configuration de l'adversaire doit demeurer secrète avant le début de la partie ;
- Le combat débute seulement lorsque les deux joueurs ont confirmé une configuration valide;
- Les joueurs jouent à tour de rôle ;
- À son tour, un joueur choisit une action parmi :
 - Attaque ;
 - Attaque puissante ;
 - Défense ;
 - Recharge.
- Le joueur transmet uniquement l'action sélectionnée ;
- Le serveur valide l'action, calcule son résultat, met à jour l'état de la partie et transmet les informations nécessaires au client ;
- Une attaque normale inflige les dégâts du robot, réduits par l'armure de l'adversaire ;
- L'attaque puissante utilise de l'énergie et ne peut être effectuée que si le joueur possède suffisamment d'énergie ;
 - Les valeurs d'énergie et de bonus devront être clairement définies et documentées.
- La Défense augmente temporairement la protection contre la prochaine attaque reçue. La Recharge permet de récupérer de l'énergie, mais remplace l'attaque du tour ;
- La partie se termine lorsqu'un robot atteint 0 HP ou moins ;
- Le client (*qu'il gagne ou perde*) peut décider de jouer une autre partie ou de quitter le jeu ;
 - Une nouvelle partie recommence par une nouvelle configuration secrète des robots.
- Le serveur rejouera également si le client veut rejouer ou se remettra en attente si le client quitte.

Protocole conceptuel (*Partiel*) - Déroulement d'une partie (à compléter)

Le diagramme suivant présente les principales étapes des échanges entre le joueur serveur et le joueur client. Il s'agit d'un protocole conceptuel :

- Les messages réels devront être définis, standardisés et sérialisés par votre équipe.
 - **Prévoir le tout dans la documentation d'équipe est plus que fortement suggéré!**

JOUEUR SERVEUR	ÉCHANGES	JOUEUR CLIENT
Lance l'application en mode serveur	← Connexion TCP	Saisit l'adresse IP et le port
Attend la connexion	Connexion établie →	Se connecte au serveur
Configure secrètement son robot		Configure secrètement son robot
Confirme sa configuration	← Configuration / PRÊT →	Confirme sa configuration
Valide les deux configurations	DÉBUT DE PARTIE →	Attend le début de la partie
Joue / valide les actions	← ACTION / RÉSULTAT →	Joue à tour de rôle
Maintient l'état officiel	MISE À JOUR →	Met à jour son affichage
Détecte la fin de partie	FIN DE PARTIE →	Reçoit le résultat
Rejoue ou retourne en attente	← REJOUER / QUITTER	Choisit de rejouer ou quitter



Principe important

- Le client ne doit pas annoncer lui-même le résultat d'une action (*par exemple le nombre de dégâts infligés*).
 - Il transmet l'action demandée, ensuite le serveur applique les règles et retourne le résultat.
- Le serveur doit également **utiliser la même logique métier** pour ses propres actions afin d'**éviter deux implémentations différentes des règles** du combat.
- Les informations privées d'un robot ne doivent pas être transmises à l'adversaire avant le moment prévu par votre protocole.